C 没有 class 怎么写泛型：void* 的真实用法

嵌入式 Linux 手记　2026-10-21　公众号文章 · 原链接 mp.weixin.qq.com

同一个 `qsort`，排 `int` 也行，排 `double` 也行，排结构体也行。库的源码里没有一个 if 在判断你的类型，C 又没有 class 和模板，它凭什么认得所有类型?

谜底藏在原型里：类型信息在门口就被没收了。

```
void qsort(void *base, size_t nmemb, size_t size,
           int (*compar)(const void *, const void *));   /* C11 §7.22.5.2 */
```

四个参数翻译成人话：数据在哪（`base`）、有几块（`nmemb`）、每块多宽（`size`）、怎么比（`compar`）。请注意里面没有出现任何具体类型——`qsort` 对你排的东西一无所知，它掌握的全部事实是"一串字节，按固定宽度切块"。这一篇拆开它：类型蒸发了，工作怎么完成。篇 12 讲过 `memcpy` 靠 `void*` 搬运一切，本篇是同一枚硬币的另一面：靠 `void*` 组织一切。

一、qsort 眼里没有类型，只有字节块

内部寻址只有一行公式：

```
第 i 个元素 = (char *)base + i * size
```

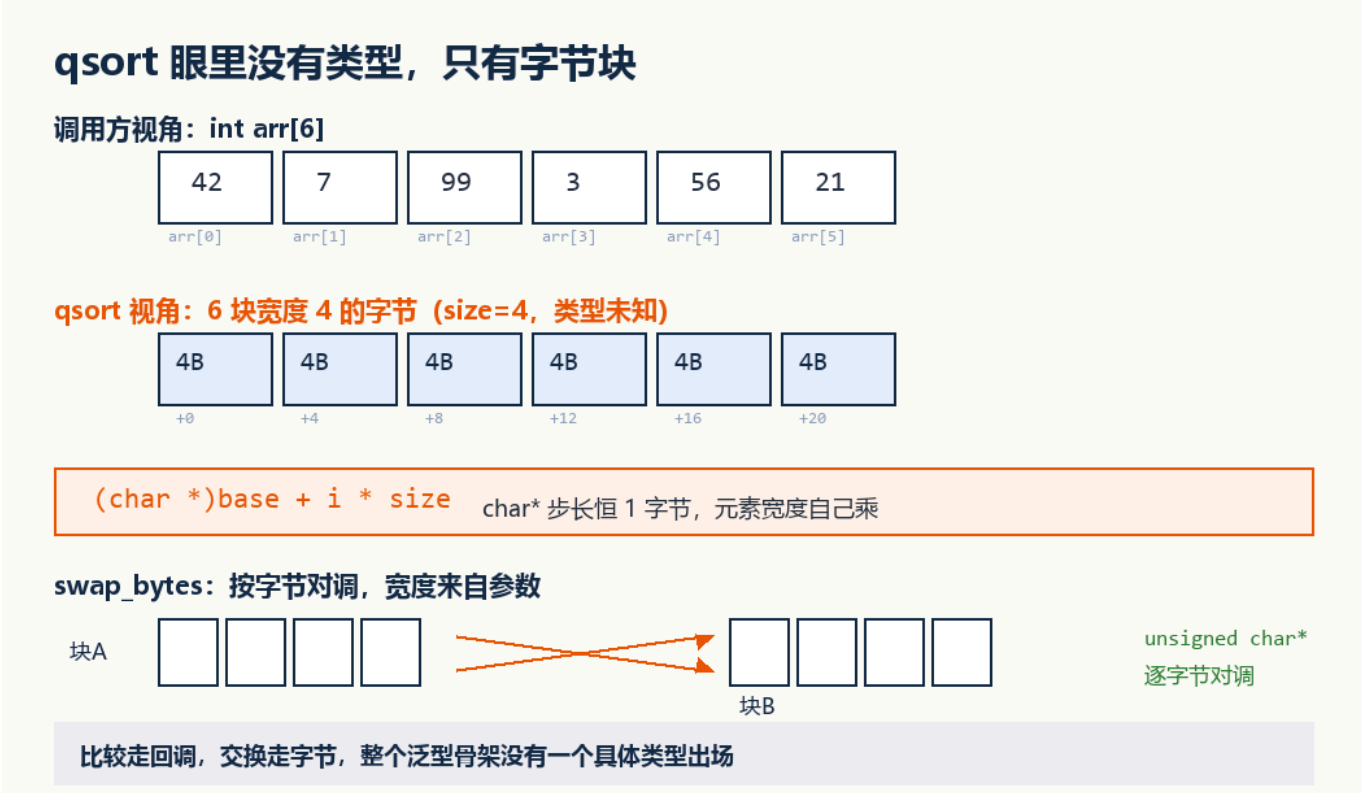
`char*` 的步长恒为 1 字节，元素宽度不能指望类型系统，只能自己乘。`int` 数组传进来，`size` 是 4，第 1 个元素在 `base + 4`；结构体数组传进来，`size` 是 `sizeof(struct T)`，公式原样成立。类型没了，算术还在。

把 `qsort` 的泛型骨架抽出来，是一个二十行的选择排序教学模型（真实 `qsort` 用更快的排序算法，但泛型骨架一模一样）：

```c
void swap_bytes(void *a, void *b, size_t size) {
    unsignedchar *pa = a, *pb = b;      /* 逐字节访问的合法通道，见第二节 */
    while (size--) {
        unsignedchar t = *pa;
        *pa++ = *pb;
        *pb++ = t;
    }
}

voidmy_qsort(void *base, size_t n, size_t size,
             int (*cmp)(const void *, const void *)) {
    char *b = base;
    for (size_t i = 0; i + 1 < n; i++) {
        size_t min = i;
        for (size_t j = i + 1; j < n; j++)
            if (cmp(b + j * size, b + min * size) < 0)
                min = j;
        if (min != i)
            swap_bytes(b + i * size, b + min * size, size);
    }
}
```

比较靠 `cmp`，交换靠 `swap_bytes`，整个函数没有一个具体类型出场。它排 `int` 也对，排 `double` 也对，排二十字节的结构体也对，因为"每个元素是什么"从来不是它的问题。



图：qsort 眼里没有类型。调用方看是 6 个 int，qsort 看是 6 块 4 字节；寻址一行公式，交换走字节。

二、unsigned char*：搬运工的合法通行证

`swap_bytes` 里那个 `unsigned char *pa = a` 不是随手一转。C11 §6.5 第 7 段规定了"表达式如何访问对象才不违反别名规则"，例外清单里有一条：任意对象的存储值，都可以用字符类型的左值读写。这是标准给"搬运工"发的通行证。不管那 4 个字节在类型系统里叫 `int` 还是 `float`，在字节搬运工眼里它们只是 4 个可读写的字节。篇 12 的 `memcpy` 内部走的也是这条通道，宽寄存器搬运的是对象表示，不是对象。

想再"聪明"一点的诱惑随之而来：既然元素是 `int`，为什么不强转 `int*` 按整字交换，非要逐字节？对齐暗雷就在这里。`(char *)base + i * size` 算出的地址只保证"字节可访问"，不保证满足 `int` 的对齐要求：元素来自任意 stride 的缓冲时，`base + 1 * size` 完全可能落在奇数地址上，`*(int *)p` 一解引用就是 UB（对齐是别名规则之外的另一条硬规矩）。所以泛型库按真实类型访问一块来路不明的内存前，标准手法是先把字节 `memcpy` 进一个对齐的临时变量再转类型。逐字节循环慢是慢，但对任何来路都成立；`qsort` 的真实实现确实会按宽度选快路径，前提是它内部先验证了对齐。库替你操心的，正是你以为不存在的细节。

三、泛型三件套：位置、宽度、行为

回头看这套机制的全部零件，只有三件：`void*` 携带位置，`size_t` 携带宽度，函数指针携带行为。比较逻辑从篇 6 就认识（回调与控制反转），搬运从篇 12 就认识（`memcpy` 与 restrict），这一篇把第三件（位置与宽度的拆分）拼上，三件套合成完整的 C 泛型。`bsearch`、`memcpy`、`memset`、`qsort` 全是这套签名的同族：`void*` 打头，宽度紧随。

它和 C++ 模板不是同行，是两种不同时代的答案：

视角	C（void* + 宽度参数）	C++ 模板
生效时机	运行期，一套代码服务所有类型	编译期按类型实例化
性能代价	间接寻址 + 比较走函数指针，较慢	无间接，但代码体积膨胀
类型检查	无（类型错了编译照样通过，运行才炸）	编译期强类型检查

`qsort` 排十种类型还是一份代码，代价是每次比较都隔着函数指针跳转、每个元素都要经过字节级搬运；模板给每种类型生成一份专用代码，比较内联成一条指令，代价是二进制膨胀与更长

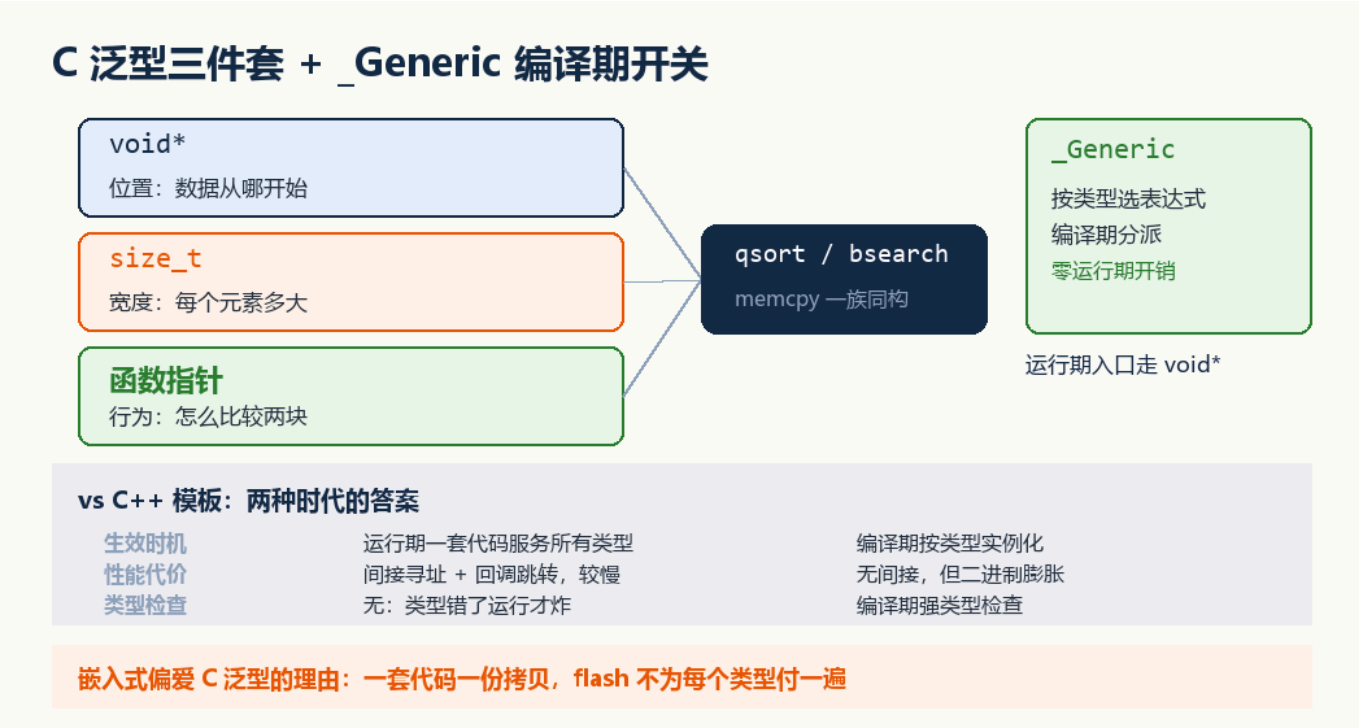
的编译时间。嵌入式选 C 泛型还有一层理由：一套代码一份拷贝，flash 不用为每个类型付一遍。

C11 补了一个编译期开关 `_Generic`（§6.5.1.1）：按表达式的类型（不是值）选择表达式，不产生任何运行期开销：

```c
#define type_name(x) _Generic((x), \
    int: "int", double: "double", char*: "char *", \
    default: "other")

printf("%s\n", type_name(3.14));   /* 打印 double */
```

分工就此清晰：运行期统一入口用 `void*` 三件套，编译期按类型分派用 `_Generic`。前者是 `qsort` 的路，后者是 `tgmath.h` 的路，两条路合起来覆盖"C 没有 class 怎么泛型"的绝大多数现场。



图：泛型三件套拼成 `qsort` 的签名；`_Generic` 补编译期分派。下方是 C 与 C++ 模板的取舍。

四、void* 的三条军规

第一条军规，出门必带宽度。`void*` 单独存在毫无信息量，不知道指向什么、也不知道多宽；解引用前必须转回真实类型（C 允许 `void*` 与对象指针隐式互转，C++ 要求显式）。看到 `void*` 形参，就去找旁边的 `size` 或文档里的宽度约定：`malloc` 返回 `void*` 但赋值目标自带类型，所以它不需要宽度；`memcpy` 收 `void*` 就必须收宽度。这是同一机制在两种接口形态下的分工。还有一层：转回的 `T*` 要按 `T` 访问，前提是那块内存的真实身份确实是 `T`（或经 `memcpy` 刚写入的有效类型），否则是严格别名违规——`void*` 只负责搬运位置，不负责洗白身份，这条规矩标准写在 C11 §6.5。

第二条军规，通道要对。读写字节用 `unsigned char*`（标准通行证）；按真实类型访问，要么保证对齐（对象数组的天然元素），要么先 `memcpy` 进对齐临时变量。顺手一提：`void *p; p + 1` 在标准 C 里是约束违反（§6.5.6 要求加法操作数指向完整对象类型），GCC 按 `sizeof(void) == 1` 收下了这写法，那是扩展不是标准。要算术，转 `char*` 再算。

第三条军规，行为注入要走函数指针。比较函数写三态返回 `(x > y) - (x < y)` 而不是相减，`INT_MIN` 与 `INT_MAX` 相遇时减法回绕会把符号送错（篇 6 定过案，此处一票通过）。泛型库把排序策略完全交给回调，回调里的 UB 由调用方自负——库连你排什么类型都不知道，自然也不知道你的比较函数藏着什么。

写在最后

1. qsort 的泛型秘密是类型信息在接口处蒸发：原型只剩 void*、宽度与回调（C11 §7.22.5.2）。
2. 内部寻址一行公式：(char *)base + i * size，char* 步长 1，宽度自己乘。
3. 字节级搬运走 unsigned char*，这是 C11 §6.5 给"访问对象表示"的合法通道；按真实类型访问来路不明的内存，先过对齐关（memcpy 进临时变量）。
4. C 泛型三件套：void* 位置、size_t 宽度、函数指针行为；运行期一套代码，代价是间接与无类型检查，_Generic 补编译期分派。
5. 军规一句话：void* 出门必带宽度、通道要对、行为注入三态返回；void* 算术是约束违反，转 char* 再算。

void* 能指万物，但 `getchar` 的返回值偏偏不能装进 `char`：256 个字符加一个"没有字符"，`char` 的格子放不下。下一篇《getchar 返回的为什么不是 char：EOF 陷阱》，看一个放错格子引发的死循环。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。